# ASSIGNMENT-02: Sensor Monitoring System (SMS)

**Embedded Linux Systems – Báo cáo kỹ thuật cuối khóa**

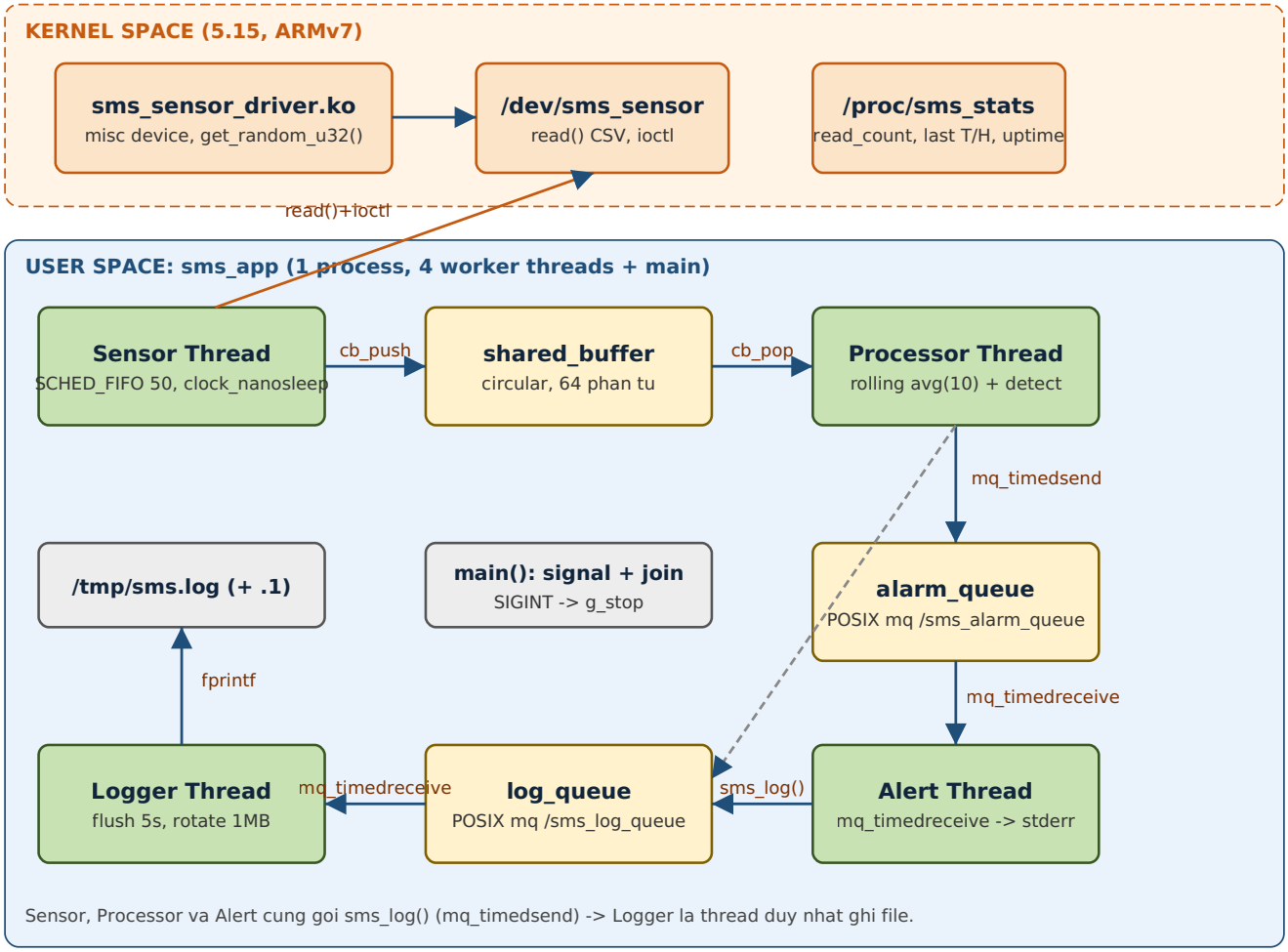
| Mục | Giá trị |
|---|---|
| Họ tên / MSSV | Hà Anh Huy / SE203732 (**kiểm tra lại trước khi nộp**) |
| Ngày thực hiện | 18/10/2026 |
| Nền tảng đích | ARMv7-A Cortex-A9, QEMU `vexpress-a9` (2 vCPU, 512 MB) |
| Kernel | Linux 5.15.0, build lại từ cây LAB-01 với `POSIX_MQUEUE`, `HIGH_RES_TIMERS`, `PREEMPT` (xem 1.5) |
| Toolchain | arm-linux-gnueabihf-gcc 13.3.0, glibc 2.39 |
| Host | Ubuntu 24.04, Linux 7.0.0-34-generic, Intel i5-13420H (12 luồng) |
| Công cụ | gdb-multiarch 15.1 + gdbserver 9.2 (ARM), Valgrind 3.22.0, perf 7.0.14, strace 6.x, stress-ng 0.17.06 |
| Mã nguồn | `driver/` (module), `app/` (6 module C + Makefile), `tools/` (script kiểm thử), `logs/` (bằng chứng) |

## Tóm tắt kết quả

| Yêu cầu | Kết quả đạt được | Bằng chứng |
|---|---|---|
| Driver `sms_sensor.ko`: read CSV, `/proc/sms_stats`, ioctl | `insmod`/`rmmod` hai lần, không oops | `logs/driver_load.txt`, `module_reload_test.txt` |
| 4 thread song song, 60 s không crash | 121 sample, 121 xử lý, 0 overflow, 0 lỗi đọc | `logs/sms_summary_stdout.txt`, `proc_status_samples.txt` |
| SIGINT dừng sạch | exit code 0, 4/4 thread `stopped`, log flush, fd đóng | `logs/sigint_test.txt` |
| Valgrind 0 definitely lost | **0 bytes in use at exit**, 19 allocs = 19 frees, **0 errors**; Helgrind: 0 race | `logs/valgrind_sms.txt`, `helgrind_sms.txt` |
| `read_count` ≈ 120 ± 10 | **121** sau 60 s với interval 500 ms | `logs/proc_sms_stats_after.txt` |
| Footprint T = 5/30/55 s | RSS cố định 2484 kB, VmSize 4652 kB | `logs/proc_status_samples.txt` |
| GDB (kịch bản A **và** B) | thực hiện cả hai trên ARM qua gdbserver | `logs/gdb_session_A.txt`, `gdb_session_B.txt` |
| strace / perf | strace trên ARM và x86; perf trên x86 | `logs/strace_sms_arm.txt`, `strace_sms.txt`, `perf_sms.txt` |
| SCHED_FIFO vs OTHER ≥ 1000 mẫu | 1500 mẫu mỗi lần; QEMU và host x86 | `logs/jitter_report*.txt` |

# 1. Thiết kế kiến trúc

## 1.1 Sơ đồ kiến trúc hệ thống




Hình 1. Kiến trúc đầy đủ: kernel module, 4 thread, cấu trúc dữ liệu chia sẻ và cơ chế IPC.

Luồng dữ liệu một mẫu: `Sensor Thread` đọc một dòng CSV từ `/dev/sms_sensor` → đẩy `SensorData` vào **circular buffer** (mutex + condition variable) → `Processor Thread` lấy ra (chờ bằng `pthread_cond_timedwait`), cập nhật rolling average 10 mẫu, chạy ba luật cảnh báo → nếu có bất thường gửi `AlertMsg` vào **POSIX message queue** `/sms_alarm_queue` → `Alert Thread` (`mq_timedreceive`) in cảnh báo ra `stderr`, tăng bộ đếm theo loại và gửi bản ghi vào `/sms_log_queue` → `Logger Thread` là thread duy nhất ghi `/tmp/sms.log`.

Thành phần theo file (mỗi file `.c` có header comment gồm mục đích, tác giả, ngày):

| File | Vai trò |
|---|---|
| `driver/sms_sensor_driver.c`, `sms_sensor.h` | Kernel module (misc device), ioctl chung với userspace |
| `app/sms_app.c` | `main()`, tham số dòng lệnh, signal, khởi tạo, tắt theo thứ tự pipeline |
| `app/sensor_thread.c/.h` | Đọc driver theo chu kỳ, đo jitter, báo cáo jitter |
| `app/processor_thread.c/.h` | Rolling average, `detect_anomalies()` |
| `app/alert_thread.c/.h` | Nhận alarm, in stderr, đếm theo loại |
| `app/logger_thread.c/.h` | Ghi log, flush 5 s, xoay tệp 1 MB |
| `app/circular_buffer.c/.h` | Bộ đệm vòng an toàn đa luồng, getter thống kê |
| `app/sms_common.h` | Kiểu dữ liệu, hằng số, biến toàn cục |

Không có đường dẫn cứng rải rác: `SMS_DEVICE_PATH`, `SMS_LOG_PATH`, tên queue là `#define` ở `sms_common.h` và có thể ghi đè bằng `-D`, `-l`. Build với `-Wall -Wextra` **không có warning** (`make all`).

## 1.2 Máy trạng thái ứng dụng

State machine

Hình 2. INIT → RUNNING → SHUTDOWN.

| Chuyển trạng thái | Điều kiện |
|---|---|
| (start) → INIT | `main()` bắt đầu; tham số sai → thoát ngay với mã 2 |
| INIT → RUNNING | Parse tham số OK, `mlockall`, 2 queue, buffer, mở driver + `ioctl(SMS_SET_INTERVAL)`, 4 thread được tạo |
| INIT → EXIT(1) | Không mở được `/dev/sms_sensor` hoặc `ioctl` lỗi, `mq_open` lỗi, không tạo được thread |
| RUNNING → SHUTDOWN | `g_stop` được đặt bởi SIGINT/SIGTERM hoặc hết `-d` giây |
| SHUTDOWN → EXIT(0) | Tất cả thread đã join, log đã flush, fd/queue đã đóng, `mq_unlink` |

Thứ tự tắt là thiết kế quan trọng: **sensor → processor → alert → logger**. Mỗi tầng chỉ dừng sau khi tầng trước đã dừng và hàng đợi của nó đã rỗng (cờ `g_sensor_done`, `g_proc_done`, `g_alert_done`, `g_logger_stop`), nên không mất mẫu hay dòng log nào còn trong pipeline. Minh chứng: lần chạy SIGINT ở `logs/sigint_test.txt` có 16 mẫu đọc = 16 mẫu xử lý = 16 produced = 16 consumed, và 3 dòng `stopped` trong log (alert/proc/sensor) cùng dòng `SUMMARY` cuối.

## 1.3 Các quyết định thiết kế quan trọng

**Circular buffer thay vì linked list.** Dung lượng cố định (64 phần tử) được cấp phát một lần lúc khởi động (`cb_create`); trong lúc chạy không có `malloc/free` ở đường dẫn nóng nên (a) thời gian `cb_push` có cận trên xác định – điều bắt buộc cho Sensor Thread chạy SCHED_FIFO; (b) không phân mảnh heap và không thể rò rỉ khi chạy nhiều ngày; (c) bộ nhớ liên tục, thân thiện cache. Linked list cần cấp phát mỗi node (thời gian không xác định, có thể gọi `brk/mmap` và page fault), và mỗi node là một chỗ có thể quên `free`. Cái giá phải trả là phải xử lý trường hợp đầy – ở đây bỏ mẫu mới, tăng `overflow_count` và ghi cảnh báo (đúng như đề yêu cầu).

**POSIX message queue cho alarm thay vì shared memory.** Alarm là sự kiện hiếm, cần đánh thức một thread đang ngủ. `mq_timedreceive` chặn trong kernel và thức dậy đúng lúc có message, kernel lo đồng bộ nên không phải tự viết thêm mutex + condvar + vòng đợi. Message có độ dài cố định (`AlertMsg` 32 byte) nên không có vấn đề framing; hàng đợi có hạn mức (10 message) giúp có back-pressure. Với shared memory phải tự thiết kế cơ chế thông báo và bảo vệ, dễ sai hơn nhiều mà không có lợi về hiệu năng ở tần suất vài alert mỗi giây. (Shared memory chỉ đáng dùng giữa các tiến trình với dữ liệu lớn.) Cùng lý do, `log_queue` cũng dùng mq: ba producer (sensor/processor/alert) gửi vào một consumer duy nhất.

**Logger Thread tách riêng.** (1) Ghi file là thao tác chặn, thời gian không xác định (flush, rotate, đĩa chậm); nếu Sensor Thread (FIFO) tự ghi log thì mọi cú trễ của I/O biến thành jitter đo cảm biến. Tách riêng làm đường dẫn nóng chỉ còn `mq_timedsend` (không chặn khi hàng đợi chưa đầy). (2) Chỉ có một thread chạm vào file nên không cần khóa, xoay tệp là một phép toán đơn luồng. (3) Cho phép gom `fflush` mỗi 5 s thay vì mỗi dòng.

**Bốn thread + main.** Main chỉ nhận tín hiệu và điều phối tắt, các thread worker chặn SIGINT/SIGTERM (`pthread_sigmask` trước khi tạo thread) nên tín hiệu luôn rơi vào main, tránh việc handler chạy ngẫu nhiên trong thread đang giữ mutex.

**Stack thread 256 KB.** Vì `mlockall(MCL_CURRENT | MCL_FUTURE)` khóa toàn bộ stack mới tạo vào RAM, stack mặc định 8 MB × 4 thread sẽ làm RSS tăng ~32 MB một cách vô ích; 256 KB là đủ cho code này và giữ footprint nhỏ (xem 3.3).

### 1.4 Thiết kế kernel driver

`sms_sensor_driver.c` là misc device (`misc_register`, minor động, node `/dev/sms_sensor` do devtmpfs tạo, mode 0666). Không có số liệu dấu phẩy động trong kernel nên nhiệt độ/độ ẩm lưu dạng ×100 và in `"%d.%02d"`.

| Chức năng | Cách làm |
|---|---|
| `read()` | Một dòng CSV `"<ts_ms>,<temp>,<hum>\n"`; `temp = 15.00 + get_random_u32() % 3001 / 100`, `hum = 30.00 + get_random_u32() % 6001 / 100`; `read_count++` dưới `sms_lock` |
| Timestamp | `ktime_to_ms(ktime_get())` – tương đương `ktime_get_ms()` mà để nhắc (hàm này **không tồn tại** trong Linux 5.15) |
| `/proc/sms_stats` | `seq_file/single_open`: `read_count`, `last_temperature`, `last_humidity`, `uptime_seconds`, `interval_ms` |
| `SMS_SET_INTERVAL` / `GET` | `_IOW('S',1,unsigned)` / `_IOR('S',2,unsigned)`, kiểm tra 1..60000 ms (mặc định 100 ms), `-EINVAL`/`-ENOTTY` đúng chuẩn |
| `O_NONBLOCK` | trả `-EAGAIN` nếu chưa đến lượt đọc |
| Tín hiệu | chờ bằng `schedule_hrtimeout` ở `TASK_INTERRUPTIBLE` → `-EINTR` khi có signal |

Hai bug thực sự của driver được tìm ra khi đo jitter (xem 3.2): (1) `msleep()` chỉ có độ phân giải 1 jiffy = 10 ms (HZ=100) làm khoảng cách giữa hai lần đọc bị kéo thành ~120 ms khi interval là 100 ms; (2) giới hạn tần suất neo vào thời điểm đọc thực làm client có cùng chu kỳ bị đẩy trễ dần (drift tích lũy). Bản cuối dùng `hrtimer` và lịch neo (`next_ok += interval`, chỉ đồng bộ lại khi client trễ ≥ 1 chu kỳ).

### 1.5 Cấu hình kernel

Kernel LAB-01 thiếu `CONFIG_POSIX_MQUEUE` (không có `mq_open`) và `CONFIG_HIGH_RES_TIMERS`. Tôi sao chép cây kernel sang `~/embedded_asgn2/kernel` (không sửa LAB-01) và đổi: `POSIX_MQUEUE=y`, `HIGH_RES_TIMERS=y`, `PREEMPT=y` (thay `PREEMPT_NONE`), giữ `HZ=100` và tick định kỳ. Thử `NO_HZ_IDLE=y` làm wake-up trong QEMU trễ rất lớn (trung bình 48 ms, do timer broadcast sp804) nên đã tắt. Tệp cấu hình: `configs/kernel_config_asgn2`; `zImage` và `.dtb` trong `output/`.

---

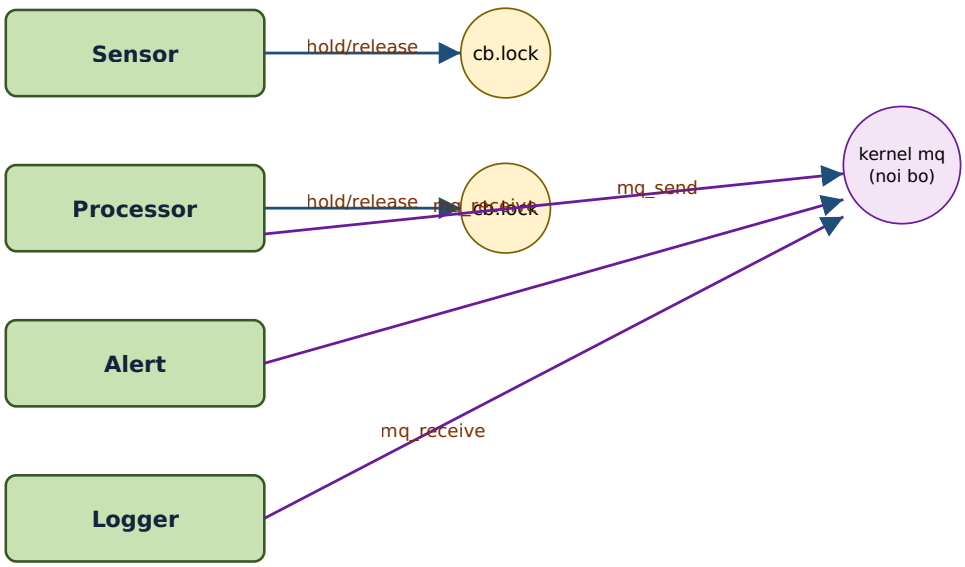
## 2. Phân tích đồng bộ hóa thread

### 2.1 Dữ liệu chia sẻ và cơ chế bảo vệ

| Dữ liệu | Ai ghi / ai đọc | Bảo vệ |
|---|---|---|
| `CircularBuffer` (data, head, tail, size, closed, 3 bộ đếm) | Sensor ghi; Processor đọc; main đọc getter | `pthread_mutex_t lock` (mọi truy cập), `not_empty` (condvar, `CLOCK_MONOTONIC`) |
| `g_stop` | Signal handler & main ghi; Sensor đọc | `atomic_int` (lock-free, an toàn trong signal handler) |
| `g_sensor_done`, `g_proc_done`, `g_alert_done`, `g_logger_stop` | Mỗi tầng ghi một lần; tầng sau đọc | `atomic_int` |
| `g_stats.*` (số mẫu, alert theo loại, lỗi đọc) | Nhiều thread tăng; main đọc | `atomic_long` (`atomic_fetch_add`) |
| `g_alarm_q`, `g_log_q` | Ghi một lần ở main trước khi tạo thread; sau đó chỉ đọc handle | Đồng bộ nội bộ của kernel (mq); handle bất biến → không cần khóa |
| Mảng jitter `g_jitter[]` | Chỉ Sensor ghi; main đọc sau `pthread_join` | `join` tạo quan hệ happens-before; không cần khóa |
| File log, bộ đếm byte, `last_flush` | Chỉ Logger | Sở hữu bởi một thread |
| Cửa sổ rolling average | Chỉ Processor | Biến cục bộ của thread |

| Dữ liệu | Ai ghi / ai đọc | Bảo vệ |
|---|---|---|
| Trạng thái driver (`read_count`, `next_ok_ns`, `interval_ms`) | Nhiều tiến trình/thread userspace | `mutex sms_lock` trong kernel |

## 2.2 Chứng minh không có deadlock




Hình 3. Resource Allocation Graph – chỉ có một mutex ứng dụng (*`cb.lock`*); các queue là đối tượng kernel.

Thứ tự acquire lock: toàn bộ chương trình chỉ có **một mutex** (`CircularBuffer.lock`) và một condvar gắn với nó. Điều kiện Coffman cần cho deadlock không thể thỏa:

1. Hold-and-wait không xảy ra: không thread nào giữ `cb.lock` khi xin tài nguyên khác. Trong vùng găng của `cb_push`, `cb_pop`, `cb_close` và các getter chỉ có phép gán/copy bộ nhớ và `pthread_cond_signal/broadcast`; **không** có `mq_*`, `read`, `sms_log`, `malloc` hay lời gọi chặn nào. `pthread_cond_timedwait` nhả `cb.lock` nguyên tử khi ngủ.
2. Do chỉ có một khóa nên không thể có chu trình thứ tự khóa (A→B và B→A). Đồ thị RAG ở Hình 3 chỉ gồm các cạnh hold/release đến một nút, không có chu trình.
3. Các chờ còn lại (queue, join) là chờ một chiều theo pipeline (sensor → processor → alert → logger), chuỗi phụ thuộc là một DAG; mọi chờ đều có timeout (`mq_timed*`, `cond_timedwait` 200 ms) nên không treo vô hạn.

Xác nhận bằng công cụ: `valgrind --tool=helgrind` trên bản x86 (15 s) cho **0 errors** (không có "lock order violated"), xem 4.2.

## 2.3 Race condition đã phát hiện và sửa trong quá trình phát triển

**Race thật số 1 – `g_stop`.** Phiên bản đầu khai báo `volatile sig_atomic_t g_stop` và dùng giữa nhiều thread. Chạy chương trình thấy bình thường, nhưng Helgrind báo:

```
Possible data race during write of size 4 at 0x1100A8 by thread #1
   at main (sms_app.c:250)
This conflicts with a previous read of size 4 by thread #5
   at sensor_thread (sensor_thread.c:191)
Address 0x1100a8 is 0 bytes inside data symbol "g_stop"
```

`volatile` chỉ cấm trình biên dịch tối ưu bỏ đọc/ghi, **không** tạo quan hệ happens-before giữa các thread (và `sig_atomic_t` chỉ bảo đảm an toàn trước signal handler trong cùng thread). Theo C11 đó là undefined behavior. Đã đổi thành `atomic_int` với `atomic_load/atomic_store` (lock-free nên vẫn dùng được trong signal handler). Chạy lại Helgrind: 0 errors.

**Lỗi tương tự (không phải race nhưng cùng loại tìm bằng công cụ):** Memcheck báo `Syscall param mq_timedsend(msg_ptr) points to uninitialised byte(s)` 215 lần – `LogMsg` và `AlertMsg` gửi qua queue chứa byte padding/đuôi chưa khởi tạo (rò rỉ nội dung stack, và kết quả không tái lập). Sửa bằng `memset` trước khi điền; log sau khi sửa: `ERROR SUMMARY: 0 errors`.

**Tình huống tiềm ẩn đã được thiết kế né:** nếu Processor thoát khi `size == 0` thì sẽ bỏ sót mẫu Sensor vừa đẩy ngay trước khi dừng. Vì vậy `cb_pop` chỉ trả `CB_CLOSED` khi **cả** `closed` **và** `size == 0` được kiểm tra trong cùng vùng găng, và Sensor gọi `cb_close()` sau lần `cb_push` cuối; kết quả đo cho thấy `produced == consumed` trong mọi lần chạy.

---

# 3. Kết quả đo lường

## 3.1 Bảng tổng hợp lần chạy 60 giây (QEMU ARM, interval 500 ms, SCHED_FIFO 50)

| Chỉ số | Giá trị |
|---|---|
| Số sample thu thập / xử lý | 121 / 121 (`read_count` driver = 121) |
| Buffer produced / consumed / overflow | 121 / 121 / **0** (tỉ lệ overflow 0 %) |
| Alert `TEMP_HIGH` (> 40 °C) | 20 |
| Alert `HUMID_HIGH` (> 80 %) | 17 |
| Alert `TEMP_SPIKE` (Δ > 5 °C) | 92 |
| Lỗi đọc / deadline hụt | 0 / 0 |
| Jitter sensor (500 ms, 121 mẫu): min / avg / max / p99 | 342 / 793 / 1088 / 1026 µs (`logs/sms_log_stats.txt`) |

Tần suất alert cao là đúng kỳ vọng thống kê: driver sinh nhiệt độ **đều** trong 15–45 °C nên P(T>40) = 1/6 (≈20/121 = 16,5 %), P(H>80) = 1/6 (14 %), và hiệu hai số ngẫu nhiên đều có |Δ| > 5 với xác suất ≈ $1-(1-5/30)^2 \approx 0{,}69$ (92/120 = 77 %).

Phân bố bản ghi trong `sms.log` (123 dòng PROC, 131 dòng ALERT = 129 alert + `started/stopped`, 2 dòng MAIN, 2 dòng SENSOR) xác nhận **tất cả bốn thread đều ghi vào log**. Sau 60 s: `/proc/sms_stats` trước/sau:

```
read_count: 0
last_temperature: 0.00
last_humidity: 0.00
uptime_seconds: 0
interval_ms: 100
```

```
read_count: 121
last_temperature: 30.73
last_humidity: 54.91
uptime_seconds: 62
interval_ms: 500
```

## 3.2 Jitter: SCHED_FIFO vs SCHED_OTHER

Phương pháp: trong Sensor Thread, ngay sau `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME)` đọc `clock_gettime` và ghi `actual − expected` (µs) vào mảng cấp phát sẵn. Chu kỳ 10 ms × 15 s ⇒ **~1500 mẫu** mỗi lần

chạy (≥ 1000). Percentile 99 = phần tử thứ ⌊0,99·(n−1)⌋ của mảng đã sắp xếp. Histogram văn bản chia 0–100 µs, 100–500 µs, 500–1000 µs, > 1 ms.

**(a) QEMU ARM (TCG, 2 vCPU)** – đo trong guest:

| Điều kiện | Policy | Mẫu | Min | Avg | Max | P99 | Hụt deadline |
|---|---|---|---|---|---|---|---|
| Rảnh | SCHED_FIFO | 1500 | 180 | 438 | 3791 | 1163 | 1 |
| Rảnh | SCHED_OTHER | ~1500 | – | 408 | 5953 | 643 | 0 |
| 2 CPU hog | SCHED_FIFO | ~1500 | – | 218 | 613 | 305 | 0 |
| 2 CPU hog | SCHED_OTHER | ~1500 | – | 279 | 2951 | 427 | 0 |
| 6 CPU hog / 2 vCPU | SCHED_FIFO | 1507 | – | 206 | 595 | 306 | 0 |
| 6 CPU hog / 2 vCPU | SCHED_OTHER | ~1500 | – | 302 | **9684** | 380 | 0 |

**(b) Host x86_64 thật** (`stress-ng --cpu 12 --io 4 --vm 2 --vm-bytes 256M`, bản x86 mô phỏng driver trong user space nên đo thuần lịch trình của scheduler):

| Policy | Mẫu | Min | Avg | Max | P99 | 0–100 µs |
|---|---|---|---|---|---|---|
| SCHED_OTHER | 1502 | 54 | 217,1 | 4948 | **2943** | 92,1 % |
| SCHED_FIFO 50 | 1502 | 4 | 7,1 | 277 | **11** | 99,9 % |

Histogram văn bản đầy đủ (QEMU):

```
ll 2>&1; cat /tmp/jit_idle_fifo.txt
=== Sensor thread wake-up jitter ===
policy   : SCHED_FIFO (priority 50)
interval : 10 ms
samples  : 1500
min_us   : 180
avg_us   : 438.1
max_us   : 3791
p99_us   : 1163
missed_deadlines (late by >= 1 period): 1
histogram (text):
  0-100us    :      0    0.0%
  100-500us  :   1299   86.6%
```

Báo cáo host (đầy đủ):

```
##### HOST x86_64 (stress-ng load) policy=other #####
=== Sensor thread wake-up jitter ===
policy   : SCHED_OTHER (priority 50)
interval : 10 ms
samples  : 1502
min_us   : 54
avg_us   : 217.1
max_us   : 4948
p99_us   : 2943
missed_deadlines (late by >= 1 period): 0
histogram (text):
  0-100us    :   1383   92.1%
  100-500us  :      4    0.3%
  500-1000us :     44    2.9%
  >1ms       :     71    4.7%
##### HOST x86_64 (stress-ng load) policy=fifo #####
=== Sensor thread wake-up jitter ===
policy   : SCHED_FIFO (priority 50)
interval : 10 ms
```

```
samples  : 1502
min_us   : 4
avg_us   : 7.1
max_us   : 277
p99_us   : 11
missed_deadlines (late by >= 1 period): 0
histogram (text):
  0-100us    :   1501   99.9%
  100-500us  :      1    0.1%
  500-1000us :      0    0.0%
  >1ms       :      0    0.0%
```

Ghi chú trung thực về QEMU: ở số liệu (a), lúc rảnh SCHED_OTHER nhỉnh hơn FIFO ở P99 (643 vs 1163 µs): khi CPU rảnh, hai vCPU vào `wfi` và cả hai policy đều chịu độ trễ đánh thức của QEMU (~0,2–0,5 ms) mà policy không ảnh hưởng; phần chênh lệch ở mức nhiễu của mô phỏng. Khi có tải, FIFO cho worst-case tốt hơn rõ rệt (max 595 vs 9684 µs với 6 hog) nhưng P99 chênh ít vì CFS ưu ái thread vừa ngủ dậy (wakeup preemption). Trên phần cứng thật (b), khác biệt là quyết định.

### 3.3 Memory footprint (QEMU, `/proc/PID/status`, chạy 60 s)

| Thời điểm | VmSize | VmRSS | VmData | VmStk | Threads |
|---|---|---|---|---|---|
| T = 5 s (khởi động) | 4652 kB | 2484 kB | 1480 kB | 132 kB | 5 |
| T = 30 s (ổn định) | 4652 kB | 2484 kB | 1480 kB | 132 kB | 5 |
| T = 55 s (trước khi kết thúc) | 4652 kB | 2484 kB | 1480 kB | 132 kB | 5 |

**RSS không tăng theo thời gian**, ba lần đo giống nhau từng kB. Nguyên nhân: (1) circular buffer và mảng jitter được cấp phát một lần lúc khởi động; (2) log không được giữ trong RAM mà đẩy qua queue có hạn mức rồi ghi file (`/tmp` là tmpfs nên dữ liệu log tính vào RAM hệ thống chứ không vào RSS của tiến trình); (3) `mlockall(MCL_CURRENT|MCL_FUTURE)` đã nạp và khóa toàn bộ trang ngay từ đầu nên cũng không còn page fault chạy dần. Dấu hiệu rò rỉ (RSS tăng đơn điệu) không xuất hiện, khớp với kết quả Valgrind.

### 3.4 `/proc/PID/maps` (T = 30 s) – giải thích 6 vùng

```
00433000-00438000 r-xp 00000000 00:02 382        /usr/bin/sms_app
00438000-00439000 r--p 00004000 00:02 382        /usr/bin/sms_app
00439000-0043a000 rw-p 00005000 00:02 382        /usr/bin/sms_app
0043a000-0045b000 rw-p 00000000 00:00 0          [heap]
76a00000-76a21000 rw-p 00000000 00:00 0
76a21000-76b00000 ---p 00000000 00:00 0
76c00000-76c21000 rw-p 00000000 00:00 0
76c21000-76d00000 ---p 00000000 00:00 0
76d86000-76d87000 ---p 00000000 00:00 0
76d87000-76dc7000 rw-p 00000000 00:00 0
76dc7000-76dc8000 ---p 00000000 00:00 0
76dc8000-76e08000 rw-p 00000000 00:00 0
76e08000-76e09000 ---p 00000000 00:00 0
76e09000-76e49000 rw-p 00000000 00:00 0
76e49000-76e4a000 ---p 00000000 00:00 0
76e4a000-76e8a000 rw-p 00000000 00:00 0
76e8a000-76f97000 r-xp 00000000 00:02 91         /lib/libc.so.6
76f97000-76f99000 r--p 0010c000 00:02 91         /lib/libc.so.6
76f99000-76f9a000 rw-p 0010e000 00:02 91         /lib/libc.so.6
76f9a000-76fa6000 rw-p 00000000 00:00 0
76fa6000-76fc2000 r-xp 00000000 00:02 90         /lib/ld-linux-armhf.so.3
76fc2000-76fc4000 r--p 0001b000 00:02 90         /lib/ld-linux-armhf.so.3
76fc4000-76fc5000 rw-p 0001d000 00:02 90         /lib/ld-linux-armhf.so.3
7e91f000-7e940000 rw-p 00000000 00:00 0          [stack]
7efe2000-7efe3000 r-xp 00000000 00:00 0          [sigpage]
7efe3000-7efe4000 r--p 00000000 00:00 0          [vvar]
```

```
7efe4000-7efe5000 r-xp 00000000 00:00 0          [vdso]
ffff0000-ffff1000 r-xp 00000000 00:00 0          [vectors]
```

| Vùng | Địa chỉ | Giải thích |
|---|---|---|
| **text** `r-xp` | `00433000-00438000 /usr/bin/sms_app` | Mã máy chương trình (20 KB); kế tiếp `r--p` (4 KB) là rodata/RELRO và `rw-p` (4 KB) là `.data/.bss` |
| **heap** | `0043a000-0045b000 [heap]` | Vùng `brk` của arena chính (132 KB): circular buffer 64×24 B, mảng jitter, `FILE*` của logger |
| **stack main** | `7e91f000-7e940000 [stack]` | 132 kB: stack của thread chính (tự giãn) |
| **stack của mỗi thread** | `76d87000-76dc7000`, `76dc8000-76e08000`, `76e09000-76e49000`, `76e4a000-76e8a000` | Mỗi thread có một vùng `mmap rw-p` đúng 256 kB (`SMS_THREAD_STACK_BYTES` = 0x40000) và ngay trước nó là một trang `---p` 4 kB là **guard page** (`76d86000`, `76dc7000`, `76e08000`, `76e49000`) – chạm vào sẽ gây SIGSEGV thay vì ghi đè vùng kế bên |
| **arena malloc phụ** | `76a00000-76a21000` + `76a21000-76b00000 ---p`; `76c00000-76c21000` + `76c21000-76d00000 ---p` | glibc cấp arena riêng cho các thread gọi `malloc`: mỗi arena dành 1 MB không gian địa chỉ (`PROT_NONE`), chỉ 132 kB đầu là `rw-p` – lý do VmSize (4652 kB) lớn hơn RSS (2484 kB) |
| **shared libs** | `76e8a000-76f97000 libc.so.6` (`r-xp` mã, `r--p` rodata, `rw-p` data), `76fa6000-76fc5000 ld-linux-armhf.so.3` | Thư viện C động (glibc 2.39, đã gồm pthread/mq/rt) và bộ nạp động |
| **vùng đặc biệt** | `[sigpage]`, `[vvar]`, `[vdso]`, `[vectors]` | Trampoline signal ARM, biến thời gian vDSO, `clock_gettime` không cần vào kernel, bảng vector ngắt `0xffff0000` |

Tổng cộng 5 stack (main + 4 thread) và 4 guard page, khớp với `Threads: 5` trong `/proc/PID/status`.

---

# 4. Task 2: Bộ nhớ và tích hợp driver

## 4.1 Valgrind – kiểm tra rò rỉ

Lệnh: `timeout -s INT 32 valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./sms_app_x86 -d 0 -p other` (chạy 32 giây rồi gửi SIGINT, bản x86 `make host`, driver mô phỏng trong user space, đúng yêu cầu "x86_64 version, ≥ 30 s, kết thúc bằng SIGINT"). Đầu ra đầy đủ: `logs/valgrind_sms.txt`.

```
==76492== Memcheck, a memory error detector
==76492== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==76492== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==76492== Command: ./sms_app_x86 -d 0 -p other -l /tmp/vg_sms.log
==76492== Parent PID: 76491
==76492==
==76492==
==76492== HEAP SUMMARY:
==76492==     in use at exit: 0 bytes in 0 blocks
==76492==   total heap usage: 19 allocs, 19 frees, 816,299 bytes allocated
==76492==
==76492== All heap blocks were freed -- no leaks are possible
==76492==
==76492== For lists of detected and suppressed errors, rerun with: -s
==76492== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Kết quả: **0 bytes definitely / indirectly / possibly lost và 0 bytes still reachable** (`in use at exit: 0 bytes in 0 blocks`), tổng 19 allocs = 19 frees (816 299 byte tổng cộng, chủ yếu là mảng jitter 100 000 × 8 B), `ERROR SUMMARY: 0 errors`. Vì không có possibly lost hay still reachable nên không cần giải thích ngoại lệ

nào. Việc "still reachable = 0" là có chủ đích: `mq_unlink`, `mq_close`, `cb_destroy`, `jitter_free` đều được gọi trên mọi đường thoát. Luồng log tự do không bị Valgrind báo nhờ mọi thread đã được join trước `main` return.

Một lưu ý: Valgrind không cho `SCHED_FIFO`/`mlockall` (chạy `-p other`, cảnh báo `mlockall` bị bỏ qua), nên kiểm tra này xác nhận logic cấp phát chứ không phải hành vi thời gian thực.

### 4.2 Helgrind – kiểm tra race / thứ tự khóa

`valgrind --tool=helgrind ./sms_app_x86 -d 15 -i 100`: **0 errors from 0 contexts** (4 173 sự kiện được Helgrind tự bỏ qua vì nằm trong nội bộ glibc/pthread). Lần chạy đầu tiên (trước khi sửa) đã bắt được race trên `g_stop`, xem 2.3.

### 4.3 Tích hợp driver

| Yêu cầu | Thực hiện | Bằng chứng |
|---|---|---|
| `open()` đường dẫn không hard-code | `cfg->device_path` từ `SMS_DEVICE_PATH` hoặc `-D` | `sms_app.c` |
| `ioctl SMS_SET_INTERVAL` khớp `-i` | `sensor_open()` gọi ioctl với `cfg->interval_ms`; `/proc/sms_stats` hiển thị `interval_ms: 500` sau khi chạy | `proc_sms_stats_after.txt` |
| `EAGAIN` | lặp lại tối đa 5 lần, nghỉ 2 ms | `sensor_thread.c: read_sample()` |
| `EINTR` | thử lại nếu chưa dừng; thoát nếu `g_stop` | như trên |
| EOF (`read()` = 0) | ghi log cảnh báo, bỏ mẫu, tiếp tục | như trên |
| Không để lọt fd | main gọi `sensor_close(fd)`; trước khi dừng có 7 fd (0,1,2, 2 queue, device, log) và sau khi thoát tiến trình biến mất (`ls /proc/PID` → `No such file`) | `sigint_test.txt` |
| `read_count` ≈ 120 ± 10 | **121** | `proc_sms_stats_after.txt` |

Kiểm tra bổ sung của driver: tháo/nạp lại hai lần (`rmmod` → mất `/dev/sms_sensor`, `insmod` → tạo lại, `read_count` về 0) và `dmesg` không có oops/BUG (`logs/module_reload_test.txt`). Mã lỗi: `-D /dev/nonexistent` → `exit_code=1`, `-i 0` và `-p bogus` → `exit_code=2` (`logs/error_codes_test.txt`).

Xoay log: chạy `-i 50 -d 12 -m 30000` cho `sms.log.1` = 30 363 byte và `sms.log` = 14 576 byte, tức tệp được đổi tên khi vượt ngưỡng và tệp mới được tạo (`logs/log_rotation_test.txt`; ngưỡng 1 MB là mặc định, `-m` chỉ để kiểm thử nhanh). Flush: mỗi `SMS_LOG_FLUSH_SEC = 5` giây thay vì mỗi dòng (`logger_thread.c`).

---

## 5. Task 3: Debug và profiling

### 5.1 GDB – cả hai kịch bản, trên ARM qua gdbserver

Môi trường: `gdbserver 0.0.0.0:1234 /usr/bin/sms_app_dbg` trong QEMU (bản `-O0 -g`, `make debug`), `gdb-multiarch` ở host với `set architecture arm`, `set sysroot`, `target remote localhost:1234`. Log đầy đủ trong `logs/gdb_session_A.txt` và `gdb_session_B.txt`, lệnh trong `gdb/scenario_A.gdb`, `scenario_B.gdb`.

**Kịch bản A – shared_buffer.** Breakpoint ở Processor Thread tại chỗ đọc `size` (`circular_buffer.c:83`):

```
Thread 2 "sms_proc" hit Breakpoint 1, cb_pop (cb=0x408190, out=0x76cfec50, timeout_ms=200) at
circular_buffer.c:83
$1 = 1   (cb->head)      $2 = 0   (cb->tail)      $3 = 1   (cb->size)
$4 = 64  (capacity)      $5 = 1   (total_produced) $6 = 0  (total_consumed)
```

Khi Sensor Thread đang ghi (`circular_buffer.c:55`), `info threads` và `thread apply all bt` cho thấy cả 5 thread:

```
  1  Thread 68.68 "sms_app_dbg"  (main)    -> __nanosleep64 ... main (sms_app.c:247)
  2  Thread 68.71 "sms_proc"     (blocked in libc, cond_timedwait)
  3  Thread 68.69 "sms_logger"   -> __clock_gettime64 ... logger_thread (logger_thread.c:62)
  4  Thread 68.70 "sms_alert"    -> __mq_timedreceive_time64 ... alert_thread (alert_thread.c:41)
* 5  Thread 68.72 "sms_sensor"   cb_push (cb=0x408190, s=0x76cbdd60) at circular_buffer.c:55
```

In `print *cb`: `head = 1, tail = 1, size = 0, total_produced = 1, total_consumed = 1` và **`lock.__data.__owner = 72` (`__nusers = 2`: Sensor đang giữ, Processor đang chờ)** – LWP của Sensor Thread – tức mutex đang thật sự được Sensor giữ trong lúc ghi, Processor phải chờ. Với `set scheduler-locking on` chỉ Sensor chạy khi `next`: `head 1→2`, `size 0→1`, `total_produced → 1…` (`$9 = 1 … $11 = 2, $12 = 1, $13 = 1`); sau khi tắt lock, Processor lấy mẫu: `head = 3, tail = 1, size = 2, total_consumed = 1`. Các bất biến `size == head − tail` (mod 64) và `produced − consumed == size` luôn đúng tại mọi điểm dừng, xác nhận vùng găng là nguyên tử với người quan sát. Vấn đề tìm được/giải thích: trong lúc Sensor giữ khóa, Processor bị chặn ở `pthread_mutex_lock` – đây là hành vi mong muốn, và vì vùng găng chỉ vài lệnh nên độ chặn tối đa nhỏ (không có inversion kéo dài).

**Kịch bản B – alert logic.** Breakpoint có điều kiện ở `detect_anomalies` (`processor_thread.c:24 if s->temperature < 30`), ngay sau khi đọc `volatile double temperature`; breakpoint thứ hai ở Alert Thread sau `mq_timedreceive` (`if n > 0`):

```
Thread 2 "sms_proc" hit Breakpoint 1, detect_anomalies (cfg=0x7efffb08, s=0x76cfec50, ...) at
processor_thread.c:24
$1 = 27.72   (s->temperature)    $2 = 61.64  (s->humidity)    $3 = 40  (cfg->temp_thr)
(gdb) set variable temperature = 45.0
$5 = 45      $6 = 1   (temperature > cfg->temp_thr)

Thread 4 "sms_alert" hit Breakpoint 2, alert_thread (arg=0x0) at alert_thread.c:42
$7 = 32      (n = sizeof(AlertMsg))
$8 = {type = 0, ts_ms = 7510, value = 45, threshold = 40}      (type 0 = ALERT_TEMP_HIGH)
```

Cùng lúc ở console QEMU: `[ALERT][7510] TYPE: TEMP_HIGH VALUE: 45.00 THRESHOLD: 40.00` (giá trị giả lập 45 °C khi nhiệt độ thật là 27,72 °C; độ ẩm 61,64 % nên không có `HUMID_HIGH`). Chuỗi sự kiện được xác nhận: Processor → `mq_timedsend` → Alert Thread thức dậy → in alert và đếm. Một điểm kỹ thuật rút ra: để `set variable` có tác dụng, giá trị phải nằm trong biến lưu trữ thật – nên `detect_anomalies` dùng bản sao `volatile`; nếu không, với `-O2` trình biên dịch sẽ giữ giá trị trong thanh ghi và lệnh GDB không tác động.

## 5.2 strace -c

**QEMU ARM, 30 s, `strace -c -f`** (`logs/strace_sms_arm.txt`, tổng 3407 syscall – bao gồm chi phí của chính strace):

| # | syscall | % time | calls | Giải thích theo logic ứng dụng |
|---|---|---|---|---|
| 1 | `clock_nanosleep` | 38,56 | 345 | Sensor Thread ngủ đến thời điểm tuyệt đối kế tiếp (TIMER_ABSTIME) + 100 ms/lần của main chờ `-d`; thời gian lớn chỉ vì ngủ (blocked time, không phải CPU) |
| 2 | `mq_timedreceive` | 34,64 | 457 (248 lỗi) | Alert và Logger chờ message; 248 `ETIMEDOUT` là các lần hết 200 ms mà không có message (cố ý để còn kiểm tra cờ thoát) |
| 3 | `futex` | 17,78 | 482 (146 lỗi) | Mutex/condvar của circular buffer; `pthread_cond_timedwait` của Processor → `FUTEX_WAIT_BITSET`, lỗi `ETIMEDOUT/EAGAIN` |
| 4 | `clock_gettime64` | 6,30 | 1666 | Mỗi lần `sms_now_ms()`, tính deadline cho `timed*`, đo jitter (strace làm vDSO bị chặn nên đếm được cả call này) |
| 5 | `mq_timedsend` | 1,17 | 209 | Processor gửi alarm; mọi thread gửi log (≈ 3 bản ghi log mỗi mẫu) |
| … | `write` / `read` | 0,44 / 0,41 | 81 / 62 | `write` = ghi `sms.log` mỗi 5 s + in `stdout/stderr`; `read` = đọc `/dev/sms_sensor` (60 lần ≈ 30 s / 500 ms) |

**Host x86_64, 30 s** (`logs/strace_sms.txt`, 1691 syscall, tổng thời gian trong syscall chỉ 0,74 s): thứ tự tương tự (`clock_nanosleep` 41,4 %, `mq_timedreceive` 37,7 %, `futex` 17,6 %, `mq_timedsend` 1,9 %, `write` 0,85 %) – phân bố giống nhau trên hai kiến trúc, chỉ khác số lần `clock_gettime` vì x86 dùng vDSO nên strace không thấy.

**User-mode vs kernel-mode.** Trên QEMU (`logs/time_sms_arm.txt`): `utime = 32` và `stime = 74` tick (CLK_TCK = 100) tức 0,32 s user so với 0,74 s kernel trong ~28 s – kernel gấp ~2,3 lần; trên host `perf stat`: `user 0,0076 s` vs `sys 0,0382 s` (gấp 5 lần), `task-clock 26 ms` trong 10,9 s tức **0,24 % CPU**, 331 context switch. Kết luận: ứng dụng **không CPU-bound**: nó gần như luôn ngủ, và phần việc thực sự là kernel call (ngủ/đánh thức, queue, futex) nên thiên về **I/O- và sự kiện-bound**.

### 5.3 perf (x86 host, `perf record -F 2000 -g`, 30 s)

Mẫu: 188 mẫu `cpu_core/cycles` (và 20 mẫu `cpu_atom`) ≈ 40 triệu chu kỳ trong 30 s, tức mẫu rất thưa vì ứng dụng nhàn rỗi. Hàm tiêu tốn nhiều nhất (`perf_sms_flat.txt`, theo comm/dso/symbol):

| # | Overhead | Thread | Object | Hàm |
|---|---|---|---|---|
| 1 | 10,29 % | sms_sensor | `sms_app_x86` | `sensor_thread` |
| 2 | 5,14 % | sms_alert | kernel | `do_mq_timedreceive` |
| 3 | 5,14 % | sms_logger | kernel | `kfree` |
| 4 | 5,14 % | sms_sensor | kernel | `switch_hrtimer_base` / `common_interrupt` |
| 5 | 5,14 % | sms_app_x86 | `ld-linux` | `_dl_fini` |

Nhận xét: 1) Hàm user-space nóng nhất là chính `sensor_thread` (vòng lặp đo jitter/`clock_gettime`/sao chép mẫu) – không có phép tính nặng. 2) **Điều bất ngờ:** phần lớn mẫu rơi vào kernel: các hàm hrtimer/scheduler (`switch_hrtimer_base`, `update_cfs_group`, `__update_load_avg_cfs_rq`, `common_interrupt`) và message queue (`do_mq_timedreceive`, `kfree` sau khi nhận message); không có hàm tính toán nào của tôi nổi lên. 3) Thấy thêm `_dl_fini` (hủy thư viện lúc thoát) – chứng tỏ vài mẫu ở giai đoạn tắt. 4) Với ~190 mẫu, mỗi mẫu = 5,14 %, nên chỉ có thể kết luận định tính (xem hạn chế ở mục 8).

**Đề xuất cải tiến dựa trên perf/strace:** (a) Số `mq_timedreceive` hết thời gian (248/457 trên ARM) và `clock_gettime` do main chờ 100 ms: thay vòng `nanosleep(100 ms)` của main bằng `sigtimedwait`/`pthread_cond_timedwait` đến thời điểm hết hạn (một lần ngủ thay vì ~600 lần thức dậy trong 60 s) và dùng `signalfd`/`eventfd` để đánh thức các queue-consumer ngay khi tắt thay vì polling 200 ms – giảm ~vài trăm context switch và công suất tiêu thụ trên thiết bị chạy pin. (b) Gộp 3 `mq_timedsend` log trên mỗi mẫu (PROC + ALERT×k) thành một bản ghi/mẫu hoặc gửi theo lô: giảm số syscall và số lần `futex` đánh thức Logger. (c) Dùng `TIMER_ABSTIME` với `timerfd`/POSIX timer cho Sensor để tránh `clock_gettime` thừa.

---

## 6. Task 4: Real-time và tối ưu

### 6.1 Triển khai

| Yêu cầu | Thực hiện |
|---|---|
| `SCHED_FIFO` priority 50 | `pthread_attr_setinheritsched(EXPLICIT)` + `setschedpolicy(SCHED_FIFO)` + priority 50 lúc `pthread_create`; trên ARM: `chrt -p` báo `SCHED_FIFO`, priority `50` cho đúng thread `sms_sensor` (`logs/proc_status_samples.txt`). Nếu không đủ quyền → cảnh báo và hạ xuống SCHED_OTHER (chạy Valgrind/host không root). |
| Lý do chọn 50 | Trên scale 1–99 của Linux, 50 cao hơn mọi thread thông thường và các thread phụ (SCHED_OTHER) nhưng chừa khoảng 49 mức cho ngắt xử lý theo thread (irq threads) và watchdog quan trọng hơn (thường 90–99). Không dùng 99 để tránh chiếm quyền của thành phần an toàn hệ thống. |

| Yêu cầu | Thực hiện |
|---|---|
| `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME)` | mốc `expected += period` được cộng dồn nên sai số không tích lũy; EINTR được thử lại cùng mốc tuyệt đối |
| `mlockall(MCL_CURRENT\|MCL_FUTURE)` | gọi ở đầu `main`; strace ghi nhận 1 lần `mlockall` thành công |
| Đo jitter ≥ 1000 mẫu, min/avg/max/P99, histogram text | xem 3.2 (1500 mẫu/lần) |
| So sánh FIFO vs OTHER | cờ `-p fifo\|other`, hai lần chạy, hai môi trường |

### 6.2 Trả lời các câu hỏi phân tích

**SCHED_FIFO có cải thiện jitter không, bao nhiêu % ở P99?** Có – **rõ rệt nhất trên phần cứng thật**: host x86 dưới tải nặng P99 giảm từ **2943 µs xuống 11 µs (−99,6 %)**, trung bình 217 → 7,1 µs (−96,7 %), max 4948 → 277 µs (−94 %), 99,9 % mẫu nằm trong 0–100 µs (so với 92,1 %). Trên QEMU mức cải thiện **nhỏ ở P99 nhưng lớn ở đuôi**: với 2 hog P99 427 → 305 µs (−28,6 %), max 2951 → 613 µs (−79 %); với 6 hog P99 380 → 306 µs (−19,5 %), max 9684 → 595 µs (−93,9 %). Lý do P99 của QEMU ít khác biệt: CFS cho thread vừa thức dậy một khoản ưu tiên (sleeper bonus + wakeup preemption), và nhiễu của mô phỏng (~0,2–0,5 ms) lớn hơn chênh lệch policy. Lúc CPU rảnh QEMU không có ưu thế FIFO (P99 FIFO 1163 µs vs OTHER 643 µs) – nhiễu mô phỏng, không phải lỗi thiết kế.

**Ngưỡng jitter tối đa chấp nhận được cho chu kỳ 500 ms?** Đọc cảm biến công nghiệp tốc độ chậm (nhiệt độ/độ ẩm) thường chịu sai số thời gian lấy mẫu cỡ vài % chu kỳ – lấy ngưỡng thiết kế **≤ 1 % chu kỳ = 5 ms** (nghiêm ngặt hơn: ≤ 0,2 % = 1 ms nếu cần đồng bộ thời gian với dữ liệu khác, hoặc nếu bộ lọc phía sau giả định lấy mẫu đều). Kết quả: max đo được trên ARM (3791 µs rảnh / 1088 µs trong lần chạy 500 ms thật) và trên host (277 µs với FIFO) **đều dưới 5 ms**, đạt; so với 1 ms thì đạt ở mọi trường hợp trừ QEMU rảnh (max 3,8 ms, p99 1,16 ms – 1,4 % mẫu > 1 ms), là nhiễu của máy ảo chứ không phải của thiết kế. Trường hợp host với SCHED_OTHER (max 4,9 ms, P99 2,9 ms) lại cho thấy nếu không dùng RT thì ngưỡng 1 ms bị vi phạm khi hệ thống tải nặng.

**Để giảm jitter thêm nữa cần thay đổi gì?** (1) **Kernel PREEMPT_RT**: ngắt xử lý trong thread, spinlock → rt_mutex có priority inheritance, giảm vùng không preempt được, giúp worst-case (đuôi) – phù hợp với kết quả "max" lớn của chúng tôi. (2) **CPU isolation**: `isolcpus=`, `nohz_full=`, `rcu_nocbs=` và `taskset/pthread_setaffinity_np` ghim Sensor vào lõi riêng, không cho thread SCHED_OTHER và ngắt khác lên lõi đó. (3) **Interrupt affinity**: `/proc/irq/*/smp_affinity` chuyển ngắt mạng/đĩa sang lõi khác. (4) Dùng `timerfd/CLOCK_TAI` với `SCHED_DEADLINE` (runtime/deadline/period khai báo, kernel kiểm tra khả năng lập lịch) thay cho FIFO thủ công. (5) Cố định tần số CPU (tắt DVFS/C-state sâu: `cpupower`, `idle=poll`) vì thức dậy từ C-state sâu thêm hàng chục µs (số liệu "rảnh" của QEMU cho thấy độ trễ khi CPU ngủ). (6) Ở phía ứng dụng: tránh `malloc`/page fault trong chu kỳ (đã `mlockall` + cấp phát trước), tránh priority inversion trên mutex dùng chung (hiện `cb.lock` được Processor SCHED_OTHER giữ vài lệnh; có thể dùng `PTHREAD_PRIO_INHERIT` cho mutex này – hiện chưa làm).

### 6.3 Hạn chế và so sánh với RT kernel thật

Không có kernel PREEMPT_RT để so sánh trực tiếp trong khuôn khổ bài; mọi nhận xét về RT chỉ là phân tích. Kernel QEMU là PREEMPT (đầy đủ preempt, không phải RT). Số liệu QEMU là thời gian ảo (TCG) nên không dùng để kết luận cho phần cứng thật.

---

## 7. Reflection – nhìn lại toàn khóa

**Kỹ năng khó nhất.** Khó nhất là debug khi mọi thứ trông bình thường: đồng bộ thread và thời gian thực. Ba lỗi thật của dự án này đều không làm chương trình crash: (i) race trên `g_stop` (chạy "đúng" hàng chục lần,

chỉ Helgrind mới thấy); (ii) 215 lỗi `mq_timedsend ... uninitialised byte(s)` chỉ Memcheck thấy; (iii) jitter trung bình 48 ms do hai bug trong driver mà ban đầu tôi nghi ở QEMU, ở timer (đã tắt NO_HZ, đã đổi HZ) rồi mới nhận ra. Cách giải quyết có hệ thống là cô lập: viết chương trình thử `jit.c` (chỉ `clock_nanosleep`, jitter 0,6 ms – chứng tỏ timer kernel tốt), rồi `jit2.c` thêm `read()` (read chiếm 9,4 ms trên chu kỳ 10 ms, jitter tăng đến 159 ms) – từ đó thấy ngay driver ép khoảng cách tối thiểu tính từ thời điểm đọc thực nên client cùng chu kỳ không bao giờ bắt kịp. Bài học: đo từng lớp một trước khi kết luận, và đừng tin trực giác "lỗi do nền tảng". Phần khó thứ hai là môi trường: không có sudo, thiếu gdbserver ARM và C++ cross-compiler nên phải tìm đường vòng (tải `.deb` giải nén cục bộ, dùng gdbserver 9.2 armhf, `gen_init_cpio` tạo device node không cần root).

**Nếu làm lại từ đầu.** (1) Thiết kế driver có giao diện "đọc theo sự kiện" (`poll()` + wait queue, tạo sample bằng hrtimer trong kernel, `read` trả mẫu mới nhất) thay vì giới hạn tần suất trong `read()` – client có thể `poll()` thay vì phải khớp chu kỳ, và hết bug drift từ gốc. (2) Dùng `timerfd` + `epoll` thay cho cặp nanosleep/thăm dò 200 ms để mọi thread chặn vô hạn và được đánh thức bằng `eventfd` khi tắt (giảm hàng trăm lần thức dậy vô ích). (3) Viết sẵn kiểm thử tự động (Valgrind/Helgrind trong Makefile target `check`) ngay từ đầu thay vì chạy tay ở cuối – hai trong ba bug nêu trên sẽ bị chặn sớm. (4) Dùng `PTHREAD_PRIO_INHERIT` cho mutex của buffer và một ring-buffer lock-free (SPSC) vì chỉ có một producer và một consumer – loại bỏ hẳn nguy cơ priority inversion. (5) Cho `LogMsg` kích thước biến đổi/ép gộp để giảm số `mq_send`.

**Kỹ năng dùng nhiều nhất trong thực tế.** Với kỹ sư Embedded Linux: đọc được trạng thái hệ thống bằng công cụ có sẵn (`/proc`, `strace -c -f`, `perf`, GDB multi-thread, Valgrind/Helgrind) và đo trước khi tối ưu – đúng như bài này: dữ liệu `perf` cho thấy ứng dụng gần như không dùng CPU, nên "tối ưu thuật toán" là sai hướng; vấn đề thực là số lần thức dậy và độ trễ lập lịch. Kỹ năng thứ hai là cross-compile + remote debug (gdbserver) vì thiết bị thật hiếm khi chạy được gdb đầy đủ. Kỹ năng thứ ba là viết kernel module nhỏ đúng chuẩn (`copy_to_user`, mutex, ioctl, procfs) và biết khi nào lỗi nằm ở kernel hay userspace.

**Cải tiến kỹ thuật muốn thêm.** Xuất dữ liệu qua MQTT (Paho C client) từ Logger Thread: mỗi mẫu đã xử lý thành một JSON gửi lên broker (`sms/<device>/telemetry`) với hàng đợi có lưu bền (persistent session) và `retain` cho trạng thái cuối, ngoài ra thêm một Web dashboard Grafana đọc qua MQTT để xem rolling average và số alert theo thời gian. Phải xử lý mất mạng (hàng đợi ngoại tuyến vào JFFS2/NAND như LAB-02) và đo ảnh hưởng của I/O mạng lên jitter Sensor Thread (đã được cách ly nhờ Logger riêng).

---

## 8. Hạn chế và trung thực về số liệu

- Dữ liệu `perf` trên x86 chỉ có ~190 mẫu vì ứng dụng nhàn rỗi (0,24 % CPU); kết luận mang tính định tính. Tôi không có `perf` trên ARM/QEMU nên `perf` chỉ chạy trên host với bản x86.
- Bản x86 mô phỏng driver trong user space (`-DSMS_SIMULATE_DRIVER`); jitter trên host đo scheduler, không đo driver. Mọi kết quả tích hợp driver (ioctl, `read_count`, `/proc/sms_stats`, `rmmod/insmod`) lấy từ bản ARM trong QEMU.
- Số liệu jitter của QEMU là thời gian trong guest dưới mô phỏng TCG; dùng để so sánh tương đối, không phải đặc tính phần cứng thật.
- Alert xảy ra thường xuyên vì dữ liệu ngẫu nhiên đều (đúng theo đề), không phản ánh dữ liệu cảm biến thật.
- Video demo là phát lại log thật với chú thích (không phải quay màn hình trực tiếp); cần quay lại nếu giảng viên yêu cầu thao tác trực tiếp.
- Họ tên/MSSV trong báo cáo được suy ra từ thư mục bài trước; cần xác nhận.

## 9. Phụ lục

**Build và chạy.**

```
cd driver && make                          # sms_sensor_driver.ko (ARMv7, vermagic 5.15.0 SMP preempt
mod_unload ARMv7)
```

```
cd app && make arm host debug             # sms_app, sms_app_x86, sms_app_dbg (-O0)
python3 tools/mkcpio.py                    # output/initramfs_asgn2.cpio.gz
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -smp 2 -nographic \
  -kernel output/zImage -dtb output/vexpress-v2p-ca9.dtb -initrd output/initramfs_asgn2.cpio.gz \
  -append "console=ttyAMA0,115200 rdinit=/sbin/init mem=512M" -nic user,model=lan9118,hostfwd=tcp::1234-:
1234
# trong QEMU:  sms_app -d 60 -v        |   cat /proc/sms_stats
tools/run_all_tests.sh                     # chay lai toan bo kiem thu tu dong
```

**Danh sách bằng chứng trong `logs/`**: `driver_load.txt`, `module_reload_test.txt`, `proc_sms_stats_before/after.txt`, `proc_status_samples.txt`, `proc_maps_T30.txt`, `sms_sample_run.txt`, `sms_alerts_stderr.txt`, `sms.log`, `sms_log_stats.txt`, `sms_summary_stdout.txt`, `sigint_test.txt`, `error_codes_test.txt`, `log_rotation_test.txt`, `valgrind_sms.txt`, `helgrind_sms.txt`, `strace_sms_arm.txt`, `strace_sms.txt`, `perf_sms.txt`, `perf_sms_flat.txt`, `perf_stat_sms.txt`, `time_sms_arm.txt`, `jitter_report.txt`, `jitter_report_heavy_load.txt`, `jitter_report_host.txt`, `gdb_session_A.txt`, `gdb_session_B.txt`, `kernel_build*.txt`.

**Tài liệu tham khảo.** Embedded Linux Primer 2nd Ed. (Hallinan) Ch. 8–19; Linux Device Drivers 3rd Ed.; The Linux Programming Interface (Kerrisk) Ch. 29–33; man `pthread_create(3)`, `mq_open(3)`, `clock_nanosleep(2)`, `sched_setscheduler(2)`; Brendan Gregg, Systems Performance 2nd Ed.; PREEMPT_RT wiki (wiki.linuxfoundation.org/realtime).